# Taller 8 — La cámara y el control de Pixel Dungeon Lite

**Sesión 8** · Corte 2 · Grupo 10 · **Individual, sobre el repositorio del equipo**

**Entrega:** al terminar la clase, un documento por estudiante (PDF o enlace) en su fila del Excel del corte, con su **usuario de GitHub** y la **URL del repositorio**. El código se revisa en sus commits.

> **Qué van a hacer.** En la demo, la cámara pasó detrás del personaje (un SpringArm3D que se maneja con el mouse) y el personaje aprendió a acelerar y frenar. Hoy **Pixel Dungeon Lite** recibe **su** cámara y su forma de tocar el mundo. Cada integrante toma **una tarea** (la cámara del juego, interactuar con lo que está cerca o apuntar con un rayo), la construye en el **repositorio del equipo** del Taller 7, en **su propio archivo**, y llena con ella un **vacío** del documento. Cada tarea tiene tres niveles; cada nivel termina en un **commit** con su usuario.
>
> **Pixel Dungeon Lite en una frase** (de su Documento de concepto): "Es Soul Knight con salas fijas y un jefe final, para jugadores que buscan retos cortos."
>
> **Es individual, pero sobre un proyecto compartido.** Cada integrante hace una tarea distinta y la escribe en **su propio archivo**. Nadie edita el archivo de otro. El documento es de cada uno; el código es de quien lo comete.
>
> **Cómo se trabaja.** Cada nivel tiene las mismas partes, en este orden. **Antes de programar:** preguntas que se responden en papel, con números; copien las respuestas al documento **antes** de escribir código. **Paso a paso:** qué hacer en el editor y en el código; hagan un paso, corran el juego (F5) y miren si pasa lo que dice **Qué deben ver**. **Esqueleto:** el código con huecos _____ que escriben ustedes. **Pistas:** en orden y solo si se traban; ninguna es la solución. **Errores típicos:** si algo raro pasa, busquen ahí el síntoma. Al final de cada nivel, **Commit**: el mensaje exacto que lleva, y **Qué va al documento**.
>
> Una predicción equivocada pero bien explicada vale más que una correcta sin explicación.

## Reparto del trabajo

| Integrante | Tarea | Archivos que crea o edita | Commits esperados |
|---|---|---|---|
| **Barriga Castro Isaac Leonardo** | A — La cámara de la sala | *camara_sigue.gd* (pegado a la Camera3D) | `T8 N1 - Tarea A - camara pegada` · `T8 N2 - Tarea A - camara con suavizado` · `T8 N3 - Tarea A - limites del nivel` |
| **Gómez Valbuena Diego Alejandro** | B — La espada (clic izquierdo) | *apuntar.gd* (nodo hijo del Personaje) y *diana.gd* | `T8 N1 - Tarea B - rayo desde el personaje` · `T8 N2 - Tarea B - rayo desde camara o mouse` · `T8 N3 - Tarea B - danio y cadencia` |
| **Mejía Poveda Bryan Smith** | C — La flecha hacia el ratón | *apuntar.gd* (nodo hijo del Personaje) y *diana.gd* | `T8 N1 - Tarea C - rayo desde el personaje` · `T8 N2 - Tarea C - rayo desde camara o mouse` · `T8 N3 - Tarea C - danio y cadencia` |

**Nadie edita el archivo de otro.** Si quieren cambiar el reparto, pueden, pero todos de acuerdo y escribiéndolo en sus documentos. **Quién crea el repositorio:** Barriga Castro Isaac Leonardo (el de la tarea A).

## Duración

| | Grupos de 3 horas | Grupos de 2 horas |
|---|---|---|
| Tiempo para el taller | 105 minutos | 45 minutos |
| Hasta dónde se espera llegar | Nivel 3 de su tarea | Nivel 2 de su tarea (el nivel 3 da acceso al 5.0) |

Tiempo sugerido en 3 horas: la base 10 min, nivel 1 20, nivel 2 35, nivel 3 30 y documento 10. En 2 horas: la base 10, nivel 1 15, nivel 2 15 y documento 5.

## Parte 1 — La base del equipo (todos, 10 minutos)

Hoy se sigue en el **mismo repositorio del Taller 7**. Antes de empezar, cada uno hace **Fetch origin** y **Pull** en GitHub Desktop.

**Qué tiene que tener su proyecto antes de empezar.** En *main.tscn*: el **Personaje** con su CollisionShape3D y su Modelo, y la **Camera3D**, hija directa de Main. En el Inspector del Personaje, **Camara** apunta a esa Camera3D. Con F5, el personaje camina. Si la tarea A del Taller 7 ya está, camina con los números del juego; si no, a 3 m/s (4 en

2.5D). Las dos cosas sirven. Si no tienen el proyecto del Taller 7, descarguen otra vez la plantilla (*Inicio del juego 3D.zip*, en la carpeta de la Sesión 7) y creen el repositorio como se hizo en el Taller 7 (lo hace **Barriga Castro Isaac Leonardo**): GitHub Desktop → *Add local repository* → *Publish*, público, y la URL en la hoja **Repositorio del juego** del Excel.

> **Cada tarea en su archivo.** La cámara se programa en un script nuevo pegado a la **Camera3D** (*camara_sigue.gd* o *camara_primera.gd*). Interactuar y apuntar se programan en un **nodo hijo** del Personaje, con su propio script (*interactuar.gd*, *apuntar.gd*), como la acción con tiempo del Taller 7. Nadie toca *personaje.gd*.

**La cuenta y los commits**, como en el Taller 7: cada uno con **su** usuario en GitHub Desktop, en su propio PC. **Un commit por nivel**, con el mensaje **exacto** que dice la guía (por ejemplo `T8 N1 - Tarea A - camara pegada`), y **Push origin** después de cada uno. Sin commit propio, el nivel se califica solo por el documento.

> **La prueba automática del Taller 7 sigue sirviendo.** En el Personaje: **Prueba** = *Avanzar 60 cuadros* para mover al personaje mientras la cámara lo sigue, y *Mantener la acción 2 s* con **Accion Prueba** = *interactuar*, *disparar* o *apuntar* para mantener la tecla de su tarea. Lo que escriben sus scripts aparece en el mismo Output.

**La tabla de números del juego** sigue en el documento de cada uno: una fila por número, si ya estaba en el documento o lo propusieron, por qué ese y lo que midió la prueba.

## Tarea A — Barriga Castro Isaac Leonardo: La cámara de la sala

Sigue la **receta A1 (una cámara que sigue)**. Tiene tres niveles: cada uno termina en un commit.

**En su juego.** Una cámara cenital sobre la sala.

**Lo que ya dice su documento. 5 salas fijas**, selladas hasta vencer a todos. Cópienlo tal cual en la tabla.

**El vacío que llena.** El tamaño de una sala. Propongan el número, escriban por qué y compruébenlo con la prueba.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a la tabla.

### Nivel 1 — Una cámara pegada al personaje

**Qué ya saben.** La Camera3D de la plantilla está **fija** en (0, 5, 7): si el personaje camina lejos, se sale de la pantalla. Una cámara que sigue se pone en la posición del personaje **más una distancia fija**: (0, 12, 4) es casi desde arriba, (6, 8, 6) es isométrica y (0, 2, 12) es de lado. Con `top_level = true` la cámara ignora a su padre y se mueve sola. `look_at_from_position(desde, hacia)` la coloca en un punto y la gira hacia otro, **una sola vez**: si se girara en cada cuadro, "adelante" cambiaría mientras el personaje camina. El Personaje usa la variable `camara` para saber qué es adelante.

**El ejemplo.** En 3D, una cámara **desde arriba**: `distancia = (0, 12, 4)`. En 2.5D, una cámara **de lado**:

`distancia = (0, 2, 12)`, con la Projection Orthogonal que ya trae la plantilla.

**Antes de programar**

1. La cámara mira al personaje desde 12 m arriba y 4 m atrás. ¿Cuántos grados queda inclinada hacia abajo? (Es el ángulo cuya tangente es 12 ÷ 4.) ¿Y la de 2.5D, con 2 m arriba y 12 atrás?
2. Con *Avanzar 60 cuadros*, el personaje termina en (0, 0, −3). ¿Dónde queda la cámara del ejemplo 3D? ¿Y la de 2.5D, con el personaje en (4, 0, 0)?
3. Para su juego: ¿qué tiene que ver el jugador alrededor del personaje? ¿Cuántos metros hacia adelante y hacia los lados? Con eso, ¿qué distancia proponen?

**Guía para la 1.** El ángulo es `atan(alto ÷ atrás)`. Con otros números: con 10 arriba y 10 atrás, atan(1) = 45°.

**Guía para la 2.** Posición del personaje más la distancia. Con otros números: personaje en (2, 0, −5) y distancia (0, 8, 6): cámara en (2, 8, 1).

**Guía para la 3.** Una cámara más alta ve más suelo, pero el personaje se ve más chico. En 2.5D, con la cámara ortográfica, **Size** es cuántos metros de alto se ven: con Size 10, de y = −2 a y = 8 si la cámara está en y = 3.

**Paso a paso**

1. Seleccionen la **Camera3D** de *main.tscn* → *Attach Script* → *camara_sigue.gd*. (Si ya tenía un script, quítenlo antes.) Ese archivo es **suyo**: nadie más lo toca.
2. Escriban el esqueleto. En el Inspector de la Camera3D: **Objetivo** = Personaje (arrastren el nodo) y **Distancia** = la del ejemplo.
3. El Personaje ya tiene en **Camara** esa misma cámara: no hay que tocar nada más.
4. F5: la cámara aparece arriba del personaje. En el Personaje, **Prueba** = *Avanzar 60 cuadros*; la línea de medida del esqueleto escribe la cámara cada medio segundo. Bórrenla al terminar.
5. A mano: caminen lejos. La cámara va pegada, sin retraso.

**Esqueleto** (*camara_sigue.gd*):

```
extends Camera3D

@export var objetivo: Node3D
@export var distancia = Vector3(0, 12, 4)


func _ready():
    top_level = true
    # Se orienta UNA vez, mirando al objetivo.
    look_at_from_position(objetivo.global_position + ____,
        objetivo.global_position)


func _physics_process(delta):
    global_position = objetivo.global_position + ____
    # Medida (bórrenla al terminar):
    if Engine.get_physics_frames() % 30 == 0:
        print("cámara ", global_position.snappedf(0.01))
```

**Qué deben ver** (ejemplo). En 3D, al empezar, la cámara está en **(0, 12, 4)**, inclinada **71.57°** hacia abajo. Con *Avanzar 60 cuadros*: cámara (0, 12, 2.6), **(0, 12, 1.1)** y (0, 12, 1.0) en las tres primeras medidas: siempre 4 m detrás y 12 arriba del personaje, que termina en z = −3. En 2.5D: (0, 2, 12), inclinada 9.46°; con el personaje caminando a 4 m/s, x = 1.87, 3.87 y **4.0**. Los cuadros del `print` cuentan desde que arranca el juego, así que pueden salir corridos en uno respecto a la prueba.

**Pistas**

- **Pista 1:** las dos veces que aparece ____ es lo mismo: la distancia.
- **Pista 2:** si al empezar la cámara está en otro lado, falta el `look_at_from_position`, que además de girarla la pone en su lugar.

**Errores típicos**

- *"Invalid access to property or key 'global_position' on a base object of type 'Nil'":* el **Objetivo** de la cámara está vacío.
- *La cámara gira con el personaje:* falta `top_level = true`, o la Camera3D quedó como hija del Personaje.
- *W lleva al personaje hacia un lado raro:* el **Camara** del Personaje apunta a otra cámara.
- *En 2.5D el personaje se ve gigante o diminuto:* es el **Size** de la cámara ortográfica, no la distancia.

**Con los números de su juego (nivel 1).** Pongan la distancia de su juego (recuadro *Con los números de su juego (nivel 2)*, más abajo) y anoten dónde queda la cámara y cuántos grados queda inclinada.

**Commit.** `T8 N1 - Tarea A - camara pegada`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y una captura de cómo se ve el juego con la cámara. El código no va: está en el commit.

## Nivel 2 — Una cámara que sigue

**Qué ya saben.** Es la cámara del nivel 1, que se orienta **una sola vez** en `_ready()`; ahora, en vez de saltar a su sitio, en cada cuadro se acerca con `lerp` a la posición del personaje **más la distancia fija**. Cambiando solo esa distancia sale cualquier cámara que sigue: (0, 12, 4) es casi desde arriba, (6, 8, 6) es isométrica y (0, 2, 12) es de lado. La cámara que sigue **no gira**: por eso "adelante" no cambia mientras el personaje camina.

**El ejemplo.** En 3D, una cámara **desde arriba**: `distancia = (0, 12, 4)` y `suavizado = 3`. En 2.5D, una cámara **de lado**: `distancia = (0, 2, 12)` y `suavizado = 4`, con la Projection Orthogonal que ya trae la plantilla.

**Antes de programar**

1. La cámara mira al personaje desde 12 m arriba y 4 m atrás. ¿Cuántos grados queda inclinada hacia abajo? (Es el ángulo cuya tangente es 12 ÷ 4.) ¿Y la de 2.5D, con 2 m arriba y 12 atrás?
2. Con `suavizado = 3`, ¿qué fracción de lo que falta avanza la cámara en un cuadro? El personaje camina 3 m hacia −Z y se detiene. En ese momento a la cámara le falta 1 m para llegar a su destino. ¿Cuánto le falta 30 cuadros después?
3. Para su juego: ¿qué tiene que ver el jugador alrededor del personaje? ¿Cuántos metros hacia adelante y hacia los lados? Con eso, ¿qué distancia proponen?

**Guía para la 1.** El ángulo es `atan(alto ÷ atrás)`. Con otros números: con 10 arriba y 10 atrás, atan(1) = 45°.

**Guía para la 2.** La fracción es `suavizado × delta`. En cada cuadro, lo que falta se multiplica por (1 − fracción). Con otros números: con `suavizado = 6` la fracción es 0.1, y de 2 m quedan $2 \times 0.9^{30} = 0.08$ m después de 30 cuadros.

**Guía para la 3.** Una cámara más alta ve más suelo, pero el personaje se ve más chico. En 2.5D, con la cámara ortográfica, **Size** es cuántos metros de alto se ven: con Size 10, de y = −2 a y = 8 si la cámara está en y = 3.

**Paso a paso**

1. Sigan en *camara_sigue.gd*, el del nivel 1: agreguen `suavizado` y cambien la línea de la posición por las dos del esqueleto (`destino` y el `lerp`).
2. En el Inspector de la Camera3D, **Suavizado** = 3. El Objetivo y la distancia ya están.
3. Prueba: en el Personaje, *Avanzar 60 cuadros*. Para ver la cámara en el Output, agreguen al final de `_physics_process()` de la cámara la línea de medida del esqueleto. Bórrenla al terminar.
4. **Con sus números.** Cambien la distancia y el suavizado por los de su juego, F5 y anoten en la tabla qué se ve.

**Esqueleto** (*camara_sigue.gd*):

```
extends Camera3D

@export var objetivo: Node3D
@export var distancia = Vector3(0, 12, 4)
@export var suavizado = 3.0


func _ready():
    top_level = true
    # Se orienta UNA vez, mirando al objetivo.
    look_at_from_position(objetivo.global_position + ____,
        objetivo.global_position)


func _physics_process(delta):
    var destino = objetivo.global_position + ____
    global_position = global_position.lerp(____, suavizado * ____)
    # Medida (bórrenla al terminar):
    if Engine.get_physics_frames() % 30 == 0:
        print("cámara ", global_position.snappedf(0.01))
```

**Qué deben ver** (ejemplo). En 3D, al empezar, la cámara está en **(0, 12, 4)**, inclinada **71.57°** hacia abajo. Con *Avanzar 60 cuadros*, el personaje termina en z = −3 y la cámara va detrás: z = 3.32 a los 30 cuadros, **2.0** a los 60, **1.22** a los 90 y 1.05 a los 120, acercándose a su destino (z = 1) sin pasarse. (Con el nivel 1 iba pegada: 2.6, 1.1 y 1.0 en esos mismos cuadros.) En 2.5D: x = 1.07 a los 30 cuadros, 2.95 a los 60, 3.87 a los 90 y 3.98 a los 120 (el personaje se detiene en x = 4), inclinada 9.46°. Los cuadros del `print` cuentan desde que arranca el juego, así que pueden salir corridos en uno.

**Pistas**

- **Pista 1:** el destino es la posición del objetivo **más** la distancia. El punto al que mira es el objetivo, sin sumar nada.
- **Pista 2:** si la cámara salta de golpe a su sitio, el `lerp` tiene como fracción 1 o más (`suavizado` de 60 o más, o falta `* delta`).
- **Pista 3:** si al empezar la cámara está en otro lado y "vuela" hacia el personaje, falta el `look_at_from_position`, que además la pone en su lugar.

**Errores típicos**

- *"Invalid access to property or key 'global_position' on a base object of type 'Nil'":* el **Objetivo** de la cámara está vacío.
- *La cámara gira con el personaje:* falta `top_level = true`, o la Camera3D quedó como hija del

Personaje.

- *W lleva al personaje hacia un lado raro:* el **Camara** del Personaje apunta a otra cámara.
- *En 2.5D el personaje se ve gigante o diminuto:* es el **Size** de la cámara ortográfica, no la distancia.

**Con los números de su juego (nivel 2).** Receta A1 con una distancia desde arriba que propongan.

**Cómo saber que funciona con sus números.** Con límites del tamaño de la sala, la cámara no muestra la sala siguiente hasta que se abre.

**Commit.** `T8 N2 - Tarea A - camara con suavizado`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

## Nivel 3 — Límites: la cámara no mira fuera del nivel

**Qué ya saben.** `clamp(valor, mínimo, máximo)`, del nivel 1 de la demo. Lo nuevo es aplicarlo al punto que la cámara sigue (el **foco**), no a la cámara: así los límites son los del nivel (los mismos números del mapa) y la distancia no importa.

**El ejemplo** (en 2.5D): la cámara no mira más allá de x = −3 ni de x = 3.

**Antes de programar**

1. El personaje camina hasta x = 4. ¿Hasta qué x llega el foco? ¿Y la cámara, con distancia (0, 2, 12)?
2. ¿Por qué se limita el foco y no la posición de la cámara? Piensen en la z de la cámara en 2.5D: está en 12. ¿Qué pasaría con un límite de z entre −10 y 10 aplicado a la cámara?
3. Para su juego: ¿cuáles son los bordes de su nivel o de su sala, en metros?

**Guía para la 1.** Con otros números: con límites de −5 a 5 y el personaje en 7, el foco queda en 5.

**Guía para la 2.** Con la distancia sumada, la cámara siempre está corrida respecto al nivel.

**Guía para la 3.** Si el nivel todavía no existe, propónganlos y anótenlos: también son números del documento.

**Paso a paso**

1. Agreguen los dos `@export` de los límites.
2. Cambien la primera línea de `_physics_process()` por las cuatro del esqueleto.
3. Prueba: *Avanzar 60 cuadros* en 2.5D y miren la x de la cámara.
4. **Con sus números:** los bordes de su nivel.

**Esqueleto.**

```
@export var limite_min = Vector2(-3, -10)   # x y z mínimas
@export var limite_max = Vector2(3, 10)     # x y z máximas

    # (al empezar _physics_process)
    var foco = objetivo.global_position
    foco.x = clamp(foco.x, limite_min.____, limite_max.____)
    foco.z = clamp(foco.z, limite_min.y, limite_max.y)
    var destino = ____ + distancia
```

**Qué deben ver** (ejemplo, 2.5D). La cámara llega a x = 2.64 a los 60 cuadros, 2.95 a los 90 y **3.0** a los 180, y no pasa de ahí aunque el personaje siga en x = 4. Su z sigue en 12.

**Pistas**

- **Pista 1:** en el `Vector2` de los límites, la x es la x del mundo y la y es la **z** del mundo.
- **Pista 2:** el destino ya no sale del objetivo, sino del foco.

**Errores típicos**

- *La cámara se fue a z = 10 y se ve todo distinto:* limitaron la cámara (el destino) y no el foco.
- *La cámara se queda quieta siempre:* el mínimo es mayor que el máximo.

**Con los números de su juego (nivel 3).** Receta A1, nivel 3: los límites de la sala.

**Commit.** `T8 N3 - Tarea A - limites del nivel`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

**Filas de su tabla de números** (agreguen las que necesiten):

| Qué es | Número | ¿Ya estaba o lo propuso? |
|---|---|---|
| Distancia de la cámara | proponer | Lo proponen (vacío) |
| Tamaño de una sala | proponer | Lo proponen (vacío) |

## Tarea B — Gómez Valbuena Diego Alejandro: La espada (clic izquierdo)

Sigue la **receta C (apuntar con un rayo)**. Tiene tres niveles: cada uno termina en un commit.

**En su juego.** El golpe de espada hacia donde mira el héroe.

**Lo que ya dice su documento.** Espada: **40 px = 1.25 m**, **1 de daño**, recarga **0.4 s**. Perseguidor **3 HP**. Cópienlo tal cual en la tabla.

**El vacío que llena.** Ninguno: comprueben que 1.25 m alcanzan al perseguidor.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a la tabla.

## Nivel 1 — Un rayo hacia donde mira el personaje

**Qué ya saben.** Un **rayo** es una línea de prueba: se le pregunta al mundo físico qué es lo primero que toca entre dos puntos, `desde` y `hacia`. La respuesta es un diccionario con `collider` (el nodo que tocó) y `position` (el punto exacto); si no toca nada, viene vacío (`is_empty()`). `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1. Un nodo hijo del Personaje tiene su propio script; con `get_parent()` llega al Personaje.

**El ejemplo:** el rayo sale del pecho del personaje (0.8 m de alto) hacia donde mira, con **20 m** de alcance. El Personaje está en (4, 0, 0), mirando hacia −Z, y la Caja1 en (4, 0.5, −4).

**Antes de programar**

1. La Caja1 mide 1 m: su cara de adelante está en z = −3.5. ¿A cuántos metros le da el rayo?
2. Si el Personaje está en (0, 0, 0) mirando hacia −Z, ¿a qué le da? (Las cajas están en (4, −4), (−5, −2) y (3, 3), en x y z.)
3. ¿Por qué hay que **excluir** al personaje del rayo, si sale de su pecho?

**Guía para la 1.** Con otros números: desde z = 2 hasta una cara en z = −1 hay 3 m.

**Guía para la 2.** Una línea desde (0, 0) hacia −Z pasa por x = 0; miren si alguna caja tiene x entre −0.5 y 0.5.

**Guía para la 3.** ¿Qué es lo primero que toca una línea que nace dentro de la cápsula del personaje?

**Paso a paso**

1. **La acción.** En el Input Map, **disparar** con la tecla de su juego (J, o *Mouse Button Left*).
2. **El nodo.** Clic derecho en **Personaje** → *Add Child Node* → **Node** → **Apuntar**. *Attach Script* → *apuntar.gd*, en la carpeta principal del proyecto. Ese archivo es **suyo**.
3. Escriban el esqueleto.
4. Para probar, pongan el Personaje en (4, 0, 0). **Prueba** = *Mantener la acción 2 s* y **Accion Prueba** = *disparar*. F5.
5. Pongan el Personaje en (0, 0, 0) y prueben otra vez. Después, a mano: caminen hacia una caja, suelten las teclas y disparen.

**Esqueleto** (*apuntar.gd*):

```
extends Node

@export var tecla = "disparar"
@export var alcance = 20.0
@onready var pj = get_parent()

var origen = Vector3.ZERO   # de dónde sale el rayo


func _physics_process(delta):
    if Input.is_action_just_pressed(tecla):
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
        else:
            var d = snappedf(origen.distance_to(golpe.____), 0.01)
            print("le dio a ", golpe.____.name, " a ", d, " m")


func lanzar_rayo():
    origen = pj.global_position + Vector3(0, 0.8, 0)
    var hacia = origen - pj.global_basis.z * ____
    var consulta = PhysicsRayQueryParameters3D.create(origen, ____)
    consulta.exclude = [pj.get_rid()]
    return pj.get_world_3d().direct_space_state.intersect_ray(consulta)
```

**Qué deben ver** (ejemplo). Desde (4, 0, 0): "**le dio a Caja1 a 3.5 m**", una sola vez aunque la tecla siga presionada. Desde (0, 0, 0): "**no le dio a nada en 20.0 m**". En 2.5D, "hacia donde mira" es +X o −X: caminando hacia la derecha, el rayo sale hacia +X.

**Pistas**

- **Pista 1:** `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1; multiplicado por el alcance, llega hasta el final del rayo.
- **Pista 2:** el diccionario del golpe tiene `collider` y `position`.
- **Pista 3:** si le da siempre al Personaje a 0 m, falta la línea de `exclude`.

**Errores típicos**

- *Siempre "no le dio a nada":* el alcance es muy corto, o el objeto no tiene CollisionShape3D (el rayo solo ve formas de colisión, no mallas).
- *Le da al Suelo en vez de a la caja:* el rayo sale muy bajo o el personaje no mira hacia la caja.
- *"Invalid access to property or key 'name'... 'Nil'":* se leyó `golpe.collider` sin preguntar antes si el golpe está vacío.

- *Dispara en todos los cuadros:* usaron `is_action_pressed` (eso es el nivel 3, con cadencia).

**Con los números de su juego (nivel 1).** Apunten a un enemigo o a un objeto de su juego (una caja con su nombre sirve) y anoten a qué distancia le da. El alcance de su juego está en el recuadro *Con los números de su juego (nivel 2)*, más abajo.

**Commit.** `T8 N1 - Tarea B - rayo desde el personaje`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

## Nivel 2 — Apuntar con un rayo

**Qué ya saben.** Del nivel 1: el rayo que sale del pecho del personaje y el diccionario del golpe (`collider` y `position`). Hay tres formas de elegir de dónde sale: **del personaje** hacia donde mira (un golpe, una espada), **del centro de la cámara** (la mira de un shooter en primera persona) o **del mouse** (un clic sobre el mundo, en un juego visto desde arriba).

**El ejemplo:** el rayo sale del pecho del personaje (0.8 m de alto) hacia donde mira, con 20 m de alcance. El Personaje está en (4, 0, 0), mirando hacia −Z, y la Caja1 en (4, 0.5, −4).

**Antes de programar**

1. La Caja1 mide 1 m: su cara de adelante está en z = −3.5. ¿A cuántos metros le da el rayo? ¿Y si el alcance es 3 m?
2. Desde el **centro de la cámara** de la plantilla (en (0, 5, 7), inclinada 35° hacia abajo), ¿a qué distancia el rayo toca el suelo? (El rayo baja 5 m; el seno de 35° es 0.574.)
3. ¿Por qué hay que **excluir** al personaje del rayo, si sale de su pecho?

**Guía para la 1.** Con otros números: desde z = 2 hasta una cara en z = −1 hay 3 m.

**Guía para la 2.** Con otros números: si la cámara está a 4 m de alto e inclinada 30° (seno 0.5), el rayo llega al suelo a 4 ÷ 0.5 = 8 m.

**Guía para la 3.** ¿Qué es lo primero que toca una línea que nace dentro de la cápsula del personaje?

**Paso a paso**

1. Sigan en *apuntar.gd*, el del nivel 1: agreguen el `@export_enum` **desde** y cambien `lanzar_rayo()` por la del esqueleto, con el `if` de las tres formas.
2. Para probar, pongan el Personaje en (4, 0, 0). **Prueba** = *Mantener la acción 2 s* y **Accion Prueba** = *disparar*. F5.
3. En el Inspector de Apuntar, cambien **Alcance** a 3 y prueben otra vez. Después, **Desde** = *Centro de la cámara*, con el Personaje en (0, 0, 0).

4. **Con sus números.** El alcance y la forma (**Desde**) de su juego.

**Esqueleto** (*apuntar.gd*):

```
extends Node

@export var tecla = "disparar"
@export var alcance = 20.0
@export_enum("Personaje", "Centro de la cámara", "Mouse") var desde = 0
@onready var pj = get_parent()

var origen = Vector3.ZERO   # de dónde sale el rayo


func _physics_process(delta):
    if Input.is_action_just_pressed(tecla):
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
        else:
            var d = snappedf(origen.distance_to(golpe.____), 0.01)
            print("le dio a ", golpe.____.name, " a ", d, " m")


func lanzar_rayo():
    var hacia
    if desde == 0:   # desde el pecho, hacia donde mira el personaje
        origen = pj.global_position + Vector3(0, 0.8, 0)
        hacia = origen - pj.global_basis.z * ____
    else:            # desde la cámara, por un punto de la pantalla
        var cam = get_viewport().get_camera_3d()
        var punto = get_viewport().get_visible_rect().size / 2
        if desde == 2:
            punto = get_viewport().get_mouse_position()
        origen = cam.project_ray_origin(punto)
        hacia = origen + cam.project_ray_normal(punto) * alcance
    var consulta = PhysicsRayQueryParameters3D.create(origen, ____)
    consulta.exclude = [pj.get_rid()]
    return pj.get_world_3d().direct_space_state.intersect_ray(consulta)
```

**Qué deben ver** (ejemplo). Desde el Personaje en (4, 0, 0): "**le dio a Caja1 a 3.5 m**". Con alcance 3: "**no le dio a nada en 3.0 m**". Desde el centro de la cámara, con el Personaje en el origen: "**le dio a Suelo a 8.72 m**". En 2.5D, "hacia donde mira" es +X o −X: caminando hacia la derecha, el rayo sale hacia +X.

**Pistas**

- **Pista 1:** `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1; multiplicado por el alcance, llega hasta el final del rayo.
- **Pista 2:** el diccionario del golpe tiene `collider` y `position`.
- **Pista 3:** si le da siempre al Personaje a 0 m, falta la línea de `exclude`.

**Errores típicos**

- *Siempre "no le dio a nada":* el alcance es muy corto, o el objeto no tiene CollisionShape3D (el rayo solo ve formas de colisión, no mallas).
- *Le da al Suelo en vez de a la caja:* el rayo sale muy bajo o el personaje no mira hacia la caja.
- *"Invalid access to property or key 'name'... 'Nil'":* se leyó `golpe.collider` sin preguntar antes si el golpe está vacío.
- *Con el mouse le da a cualquier cosa:* **Desde** quedó en *Centro de la cámara*.

**Con los números de su juego (nivel 2).** Receta C desde el Personaje con **espada** (*Mouse Button Left*) y `alcance = 1.25`.

**Cómo saber que funciona con sus números.** El perseguidor cae en el tercer golpe, en el cuadro 49.

**Commit.** `T8 N2 - Tarea B - rayo desde camara o mouse`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

## Nivel 3 — Golpe con daño y cadencia

**Qué ya saben.** La recarga del Taller 7, ahora como **cadencia**: un tiempo mínimo entre un golpe y el siguiente. Y `has_method`: si lo que tocó el rayo sabe recibir golpes, se le pide que reciba uno.

**El ejemplo:** manteniendo la tecla, un golpe cada **0.25 s** con **25 de daño**. La Caja1 tiene el script *diana.gd*, con **100 de vida**.

**Antes de programar**

1. ¿Cuántos golpes necesita para quedar en 0? ¿En qué cuadros pega, si el primero es en el cuadro 1? ¿Cuánto tiempo pasa entre el primero y el último?
2. Después de destruirla, ¿qué escribe el rayo en el siguiente golpe?
3. Para su juego: daño, cadencia y vida del enemigo (del recuadro de su juego; lo que falte, propónganlo).

**Guía para la 1.** La cadencia en cuadros es 0.25 × 60 = 15. Con otros números: con 0.5 s y 3 golpes, pega en los cuadros 1, 31 y 61.

**Guía para la 2.** `queue_free()` saca el nodo del mundo.

**Guía para la 3.** Con vida 3 y daño 1, son 3 golpes; con la cadencia, eso da el tiempo que tarda en vencerlo.

**Paso a paso**

1. Agreguen `danio`, `cadencia` y `espera`, y cambien `_physics_process()` por el del esqueleto 1.
2. A la Caja1, *Attach Script* → *diana.gd* (esqueleto 2). En su juego, la función `recibir_golpe()` va en el script del enemigo (el de la receta C del Taller 7).
3. Prueba: Personaje en (4, 0, 0), *Mantener la acción 2 s* con *disparar*.

**Esqueleto 1** (*apuntar.gd*):

```
@export var danio = 25
@export var cadencia = 0.25

var espera = 0.0


func _physics_process(delta):
    espera -= delta
    if Input.is_action_pressed(tecla) and espera <= 0.001:
        espera = ____
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
            return
        var d = snappedf(origen.distance_to(golpe.position), 0.01)
        print("le dio a ", golpe.collider.name, " a ", d, " m")
        if golpe.collider.has_method("recibir_golpe"):
            golpe.collider.recibir_golpe(____)
```

**Esqueleto 2** (*diana.gd*):

```
extends StaticBody3D

@export var vida = 100


func recibir_golpe(danio):
    vida -= ____
    print(name, ": vida ", vida)
    if vida <= 0:
        print(name, " destruida")
        queue_free()
```

**Qué deben ver** (ejemplo). "le dio a Caja1 a 3.5 m" y "Caja1: vida 75" (cuadro 1), 50 (cuadro 16), 25 (cuadro 31), 0 y "**Caja1 destruida**" (cuadro **46**, 0.75 s después del primero). Desde ahí, cada 15 cuadros: "no le dio a nada en 20.0 m".

**Pistas**

- **Pista 1:** `espera` cuenta hacia atrás como el `tiempo` del Taller 7; se recarga con la cadencia en cada golpe.
- **Pista 2:** si la diana no pierde vida, el nombre de la función no es exactamente `recibir_golpe`.

**Errores típicos**

- *Pega en todos los cuadros:* falta `espera = cadencia`.
- *Solo pega una vez:* quedó `is_action_just_pressed`.
- *"Invalid call. Nonexistent function 'recibir_golpe'":* llamaron la función sin preguntar con `has_method`, y le dio al Suelo.

**Con los números de su juego (nivel 3).** Receta C, nivel 3, con `danio = 1` y `cadencia = 0.4`; la diana es el perseguidor, con `vida = 3`.

**Commit.** `T8 N3 - Tarea B - danio y cadencia`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

**Filas de su tabla de números** (agreguen las que necesiten):

| Qué es | Número | ¿Ya estaba o lo propuso? |
|---|---|---|
| Alcance | 1.25 m | Ya estaba en el documento |
| Daño | 1 | Ya estaba en el documento |
| Recarga | 0.4 s | Ya estaba en el documento |
| Vida del perseguidor | 3 | Ya estaba en el documento |

## Tarea C — Mejía Poveda Bryan Smith: La flecha hacia el ratón

Sigue la **receta C (apuntar con un rayo)**. Tiene tres niveles: cada uno termina en un commit.

**En su juego.** Disparar una flecha hacia donde apunta el ratón.

**Lo que ya dice su documento.** Flecha (clic derecho): **2 de daño**, **1 munición**, recarga **0.6 s**. Munición inicial **5** (máx. 10). Cópienlo tal cual en la tabla.

**El vacío que llena. El alcance de la flecha.** Propongan el número, escriban por qué y compruébenlo con la prueba.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a

la tabla.

## Nivel 1 — Un rayo hacia donde mira el personaje

**Qué ya saben.** Un **rayo** es una línea de prueba: se le pregunta al mundo físico qué es lo primero que toca entre dos puntos, `desde` y `hacia`. La respuesta es un diccionario con `collider` (el nodo que tocó) y `position` (el punto exacto); si no toca nada, viene vacío (`is_empty()`). `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1. Un nodo hijo del Personaje tiene su propio script; con `get_parent()` llega al Personaje.

**El ejemplo:** el rayo sale del pecho del personaje (0.8 m de alto) hacia donde mira, con **20 m** de alcance. El Personaje está en (4, 0, 0), mirando hacia −Z, y la Caja1 en (4, 0.5, −4).

**Antes de programar**

1. La Caja1 mide 1 m: su cara de adelante está en z = −3.5. ¿A cuántos metros le da el rayo?
2. Si el Personaje está en (0, 0, 0) mirando hacia −Z, ¿a qué le da? (Las cajas están en (4, −4), (−5, −2) y (3, 3), en x y z.)
3. ¿Por qué hay que **excluir** al personaje del rayo, si sale de su pecho?

**Guía para la 1.** Con otros números: desde z = 2 hasta una cara en z = −1 hay 3 m.

**Guía para la 2.** Una línea desde (0, 0) hacia −Z pasa por x = 0; miren si alguna caja tiene x entre −0.5 y 0.5.

**Guía para la 3.** ¿Qué es lo primero que toca una línea que nace dentro de la cápsula del personaje?

**Paso a paso**

1. **La acción.** En el Input Map, **disparar** con la tecla de su juego (J, o *Mouse Button Left*).
2. **El nodo.** Clic derecho en **Personaje** → *Add Child Node* → **Node** → **Apuntar**. *Attach Script* → *apuntar.gd*, en la carpeta principal del proyecto. Ese archivo es **suyo**.
3. Escriban el esqueleto.
4. Para probar, pongan el Personaje en (4, 0, 0). **Prueba** = *Mantener la acción 2 s* y **Accion Prueba** = *disparar*. F5.
5. Pongan el Personaje en (0, 0, 0) y prueben otra vez. Después, a mano: caminen hacia una caja, suelten las teclas y disparen.

**Esqueleto** (*apuntar.gd*):

```
extends Node

@export var tecla = "disparar"
@export var alcance = 20.0
@onready var pj = get_parent()

var origen = Vector3.ZERO   # de dónde sale el rayo


func _physics_process(delta):
    if Input.is_action_just_pressed(tecla):
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
        else:
            var d = snappedf(origen.distance_to(golpe.____), 0.01)
            print("le dio a ", golpe.____.name, " a ", d, " m")


func lanzar_rayo():
    origen = pj.global_position + Vector3(0, 0.8, 0)
    var hacia = origen - pj.global_basis.z * ____
    var consulta = PhysicsRayQueryParameters3D.create(origen, ____)
    consulta.exclude = [pj.get_rid()]
    return pj.get_world_3d().direct_space_state.intersect_ray(consulta)
```

**Qué deben ver** (ejemplo). Desde (4, 0, 0): "**le dio a Caja1 a 3.5 m**", una sola vez aunque la tecla siga presionada. Desde (0, 0, 0): "**no le dio a nada en 20.0 m**". En 2.5D, "hacia donde mira" es +X o −X: caminando hacia la derecha, el rayo sale hacia +X.

**Pistas**

- **Pista 1:** `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1; multiplicado por el alcance, llega hasta el final del rayo.
- **Pista 2:** el diccionario del golpe tiene `collider` y `position`.
- **Pista 3:** si le da siempre al Personaje a 0 m, falta la línea de `exclude`.

**Errores típicos**

- *Siempre "no le dio a nada":* el alcance es muy corto, o el objeto no tiene CollisionShape3D (el rayo solo ve formas de colisión, no mallas).
- *Le da al Suelo en vez de a la caja:* el rayo sale muy bajo o el personaje no mira hacia la caja.
- *"Invalid access to property or key 'name'... 'Nil'":* se leyó `golpe.collider` sin preguntar antes si el golpe está vacío.

- *Dispara en todos los cuadros:* usaron `is_action_pressed` (eso es el nivel 3, con cadencia).

**Con los números de su juego (nivel 1).** Apunten a un enemigo o a un objeto de su juego (una caja con su nombre sirve) y anoten a qué distancia le da. El alcance de su juego está en el recuadro *Con los números de su juego (nivel 2)*, más abajo.

**Commit.** `T8 N1 - Tarea C - rayo desde el personaje`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

## Nivel 2 — Apuntar con un rayo

**Qué ya saben.** Del nivel 1: el rayo que sale del pecho del personaje y el diccionario del golpe (`collider` y `position`). Hay tres formas de elegir de dónde sale: **del personaje** hacia donde mira (un golpe, una espada), **del centro de la cámara** (la mira de un shooter en primera persona) o **del mouse** (un clic sobre el mundo, en un juego visto desde arriba).

**El ejemplo:** el rayo sale del pecho del personaje (0.8 m de alto) hacia donde mira, con 20 m de alcance. El Personaje está en (4, 0, 0), mirando hacia −Z, y la Caja1 en (4, 0.5, −4).

**Antes de programar**

1. La Caja1 mide 1 m: su cara de adelante está en z = −3.5. ¿A cuántos metros le da el rayo? ¿Y si el alcance es 3 m?
2. Desde el **centro de la cámara** de la plantilla (en (0, 5, 7), inclinada 35° hacia abajo), ¿a qué distancia el rayo toca el suelo? (El rayo baja 5 m; el seno de 35° es 0.574.)
3. ¿Por qué hay que **excluir** al personaje del rayo, si sale de su pecho?

**Guía para la 1.** Con otros números: desde z = 2 hasta una cara en z = −1 hay 3 m.

**Guía para la 2.** Con otros números: si la cámara está a 4 m de alto e inclinada 30° (seno 0.5), el rayo llega al suelo a 4 ÷ 0.5 = 8 m.

**Guía para la 3.** ¿Qué es lo primero que toca una línea que nace dentro de la cápsula del personaje?

**Paso a paso**

1. Sigan en *apuntar.gd*, el del nivel 1: agreguen el `@export_enum` **desde** y cambien `lanzar_rayo()` por la del esqueleto, con el `if` de las tres formas.
2. Para probar, pongan el Personaje en (4, 0, 0). **Prueba** = *Mantener la acción 2 s* y **Accion Prueba** = *disparar*. F5.
3. En el Inspector de Apuntar, cambien **Alcance** a 3 y prueben otra vez. Después, **Desde** = *Centro de la cámara*, con el Personaje en (0, 0, 0).

4. **Con sus números.** El alcance y la forma (**Desde**) de su juego.

**Esqueleto** (*apuntar.gd*):

```
extends Node

@export var tecla = "disparar"
@export var alcance = 20.0
@export_enum("Personaje", "Centro de la cámara", "Mouse") var desde = 0
@onready var pj = get_parent()

var origen = Vector3.ZERO   # de dónde sale el rayo


func _physics_process(delta):
    if Input.is_action_just_pressed(tecla):
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
        else:
            var d = snappedf(origen.distance_to(golpe.____), 0.01)
            print("le dio a ", golpe.____.name, " a ", d, " m")


func lanzar_rayo():
    var hacia
    if desde == 0:   # desde el pecho, hacia donde mira el personaje
        origen = pj.global_position + Vector3(0, 0.8, 0)
        hacia = origen - pj.global_basis.z * ____
    else:            # desde la cámara, por un punto de la pantalla
        var cam = get_viewport().get_camera_3d()
        var punto = get_viewport().get_visible_rect().size / 2
        if desde == 2:
            punto = get_viewport().get_mouse_position()
        origen = cam.project_ray_origin(punto)
        hacia = origen + cam.project_ray_normal(punto) * alcance
    var consulta = PhysicsRayQueryParameters3D.create(origen, ____)
    consulta.exclude = [pj.get_rid()]
    return pj.get_world_3d().direct_space_state.intersect_ray(consulta)
```

**Qué deben ver** (ejemplo). Desde el Personaje en (4, 0, 0): "**le dio a Caja1 a 3.5 m**". Con alcance 3: "**no le dio a nada en 3.0 m**". Desde el centro de la cámara, con el Personaje en el origen: "**le dio a Suelo a 8.72 m**". En 2.5D, "hacia donde mira" es +X o −X: caminando hacia la derecha, el rayo sale hacia +X.

**Pistas**

- **Pista 1:** `-pj.global_basis.z` es hacia donde mira el personaje, de largo 1; multiplicado por el alcance, llega hasta el final del rayo.
- **Pista 2:** el diccionario del golpe tiene `collider` y `position`.
- **Pista 3:** si le da siempre al Personaje a 0 m, falta la línea de `exclude`.

**Errores típicos**

- *Siempre "no le dio a nada":* el alcance es muy corto, o el objeto no tiene CollisionShape3D (el rayo solo ve formas de colisión, no mallas).
- *Le da al Suelo en vez de a la caja:* el rayo sale muy bajo o el personaje no mira hacia la caja.
- *"Invalid access to property or key 'name'... 'Nil'":* se leyó `golpe.collider` sin preguntar antes si el golpe está vacío.
- *Con el mouse le da a cualquier cosa:* **Desde** quedó en *Centro de la cámara*.

**Con los números de su juego (nivel 2).** Receta C con **Desde = Mouse** y **flecha** (*Mouse Button Right*), con el alcance que propongan.

**Cómo saber que funciona con sus números.** Con 5 flechas, el sexto disparo no sale. Un perseguidor de 3 HP cae con 2 flechas.

**Commit.** `T8 N2 - Tarea C - rayo desde camara o mouse`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

## Nivel 3 — Golpe con daño y cadencia

**Qué ya saben.** La recarga del Taller 7, ahora como **cadencia**: un tiempo mínimo entre un golpe y el siguiente. Y `has_method`: si lo que tocó el rayo sabe recibir golpes, se le pide que reciba uno.

**El ejemplo:** manteniendo la tecla, un golpe cada **0.25 s** con **25 de daño**. La Caja1 tiene el script *diana.gd*, con **100 de vida**.

**Antes de programar**

1. ¿Cuántos golpes necesita para quedar en 0? ¿En qué cuadros pega, si el primero es en el cuadro 1? ¿Cuánto tiempo pasa entre el primero y el último?
2. Después de destruirla, ¿qué escribe el rayo en el siguiente golpe?
3. Para su juego: daño, cadencia y vida del enemigo (del recuadro de su juego; lo que falte, propónganlo).

**Guía para la 1.** La cadencia en cuadros es 0.25 × 60 = 15. Con otros números: con 0.5 s y 3 golpes, pega en los cuadros 1, 31 y 61.

**Guía para la 2.** `queue_free()` saca el nodo del mundo.

**Guía para la 3.** Con vida 3 y daño 1, son 3 golpes; con la cadencia, eso da el tiempo que tarda en vencerlo.

**Paso a paso**

1. Agreguen `danio`, `cadencia` y `espera`, y cambien `_physics_process()` por el del esqueleto 1.
2. A la Caja1, *Attach Script* → *diana.gd* (esqueleto 2). En su juego, la función `recibir_golpe()` va en el script del enemigo (el de la receta C del Taller 7).
3. Prueba: Personaje en (4, 0, 0), *Mantener la acción 2 s* con *disparar*.

**Esqueleto 1** (*apuntar.gd*):

```
@export var danio = 25
@export var cadencia = 0.25

var espera = 0.0


func _physics_process(delta):
    espera -= delta
    if Input.is_action_pressed(tecla) and espera <= 0.001:
        espera = ____
        var golpe = lanzar_rayo()
        if golpe.is_empty():
            print("no le dio a nada en ", alcance, " m")
            return
        var d = snappedf(origen.distance_to(golpe.position), 0.01)
        print("le dio a ", golpe.collider.name, " a ", d, " m")
        if golpe.collider.has_method("recibir_golpe"):
            golpe.collider.recibir_golpe(____)
```

**Esqueleto 2** (*diana.gd*):

```
extends StaticBody3D

@export var vida = 100


func recibir_golpe(danio):
    vida -= ____
    print(name, ": vida ", vida)
    if vida <= 0:
        print(name, " destruida")
        queue_free()
```

**Qué deben ver** (ejemplo). "le dio a Caja1 a 3.5 m" y "Caja1: vida 75" (cuadro 1), 50 (cuadro 16), 25 (cuadro 31), 0 y "**Caja1 destruida**" (cuadro **46**, 0.75 s después del primero). Desde ahí, cada 15 cuadros: "no le dio a nada en 20.0

m".

**Pistas**

- **Pista 1:** `espera` cuenta hacia atrás como el `tiempo` del Taller 7; se recarga con la cadencia en cada golpe.
- **Pista 2:** si la diana no pierde vida, el nombre de la función no es exactamente `recibir_golpe`.

**Errores típicos**

- *Pega en todos los cuadros:* falta `espera = cadencia`.
- *Solo pega una vez:* quedó `is_action_just_pressed`.
- *"Invalid call. Nonexistent function 'recibir_golpe'":* llamaron la función sin preguntar con `has_method`, y le dio al Suelo.

**Con los números de su juego (nivel 3).** Receta C, nivel 3, con `danio = 2` y `cadencia = 0.6`, y la munición: sin flechas no dispara.

**Commit.** `T8 N3 - Tarea C - danio y cadencia`, y *Push origin*.

**Qué va al documento.** Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

**Filas de su tabla de números** (agreguen las que necesiten):

| Qué es | Número | ¿Ya estaba o lo propuso? |
|---|---|---|
| Daño | 2 | Ya estaba en el documento |
| Recarga | 0.6 s | Ya estaba en el documento |
| Munición inicial | 5 | Ya estaba en el documento |
| Alcance de la flecha | proponer | Lo proponen (vacío) |

## Extra — Para quien termine todo

1. **La tarea de al lado.** El nivel 1 y el 2 de otra tarea de esta guía, en un archivo aparte, sin tocar el de su compañero. Con su commit: `T8 N1 - Tarea X - extra`.
2. **El kart y el runner de la demo.** En la Base Taller 8 están *pista_kart.tscn* y *pista_runner.tscn*; sus recetas están en la guía anterior de esta sesión (*Tres formas de controlar*), que les comparte el profesor. Sirven para los juegos con vehículo o con carriles.

## Documento de entrega

Un documento por estudiante (PDF o enlace), en su fila de la hoja del taller en el Excel del corte. Trae, en este orden:

1. **Nombre completo**, **usuario de GitHub** (el que aparece en sus commits) y la **URL del repositorio** del equipo.
2. **Su tarea** (A, B o C, con su receta) y el nombre de su archivo.
3. **Sus commits:** por cada nivel, el mensaje y el enlace al commit (en GitHub, *Commits* → el commit → copiar la URL).
4. **Por cada nivel:** lo que pide **Qué va al documento**. El código **no** se copia: está en el commit.
5. **La tabla de números del juego**, completa.
6. Respuestas cortas, con sus palabras:
7. **a.** ¿Qué hace el SpringArm3D de la demo que no hace una Camera3D hija del personaje?
8. **b.** ¿Por qué la cámara que sigue (o la de los ojos) se orienta una sola vez y no en cada cuadro mirando al personaje?
9. **c.** ¿Qué vacío del documento llenó su tarea? ¿Qué número propuso y por qué ese y no otro?
10. **d.** Cuando lo probó en el juego, ¿el número se sintió bien? Usen la medida de la prueba para decir si lo dejaría así o lo cambiaría, y a cuánto.
11. **Anexo IA:** si usaron IA, copien cada prompt que escribieron y la respuesta completa que obtuvieron. Si no la usaron, escriban "No se usó IA". Si usaron IA y no hay anexo, el taller no se califica.

## Cómo se califica (0.0 a 5.0)

Cada nivel se califica en **su commit** (el código corre y da lo que dice *Qué deben ver*, con el ejemplo y con sus números) más lo del documento. Un nivel sin commit con su usuario se califica solo por el documento, y vale menos. Cada franja exige todo lo de la anterior; si una franja queda a medias, la nota queda entre las dos (por ejemplo, 3.5 o 4.5).

| Nota | Grupos de 3 horas | Grupos de 2 horas |
|---|---|---|
| **3.0** | Nivel 1 de su tarea: el commit, las pruebas con el ejemplo comparadas con *Qué deben ver* y con sus números, y las respuestas de *Antes de programar*. | Nivel 1 de su tarea: el commit y las pruebas con el ejemplo. |
| **4.0** | Nivel 2 de su tarea: el commit, las pruebas con el ejemplo y con sus números, y la tabla de números con el vacío que llenó y su porqué. Respuestas a, b y c. | Nivel 2 de su tarea: el commit, la prueba con el ejemplo y con su número, y la tabla con el vacío que llenó. Respuestas a y c. |
| **5.0** | Nivel 3 de su tarea: el commit, la medida comparada con su predicción y la tabla completa. Respuesta d. | Nivel 3 de su tarea: el commit y la tabla, aunque falte una de las pruebas. Respuesta d. |

Hacer bien lo pedido hasta el nivel 2 da 4.0. El 5.0 es para quien resuelve y explica el nivel 3 de su tarea. El documento sin usuario de GitHub o sin la URL del repositorio no se califica hasta que los tenga.